

## Задание 7. Отчет об аудите безопасности

### Раздел 1. Уязвимости XSS.

Для того, чтобы никто не мог вставить вредоносный скрипт в страницу, который выполняется в браузере пользователя, была добавлена фильтрация вывода. Все динамические данные, выводимые из базы данных, проходят через фильтрацию функцией `htmlspecialchars()`. Это преобразует спецсимволы в безопасный текст.

#### Пример кода:

```
<td><?php echo htmlspecialchars($user['fio']); ?></td>  
<td><?php echo htmlspecialchars($user['email']); ?></td>  
<td><?php echo htmlspecialchars($user['phone']); ?></td>
```

### Раздел 2. Information Disclosure.

База была настроена так, чтобы при ошибках не происходила утечка технической информации о системе. Режим работы PDO был переведен на «тихий» (`ERRMODE_SILENT`). Ошибки не появятся на экране обычного пользователя.

#### Пример кода:

```
$db->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_SILENT);
```

### Раздел 3. SQL Injection.

Защита базы данных от взлома через SQL запросы. Был использован механизм подготовленных запросов (Prepared Statements) через расширение PDO. Так данные и сам запрос идут отдельно, поэтому исключена возможность изменения структуры SQL-запроса.

#### Пример кода:

```
$stmt = $db->prepare("SELECT password_hash FROM admins WHERE login = ?");  
$stmt->execute([$$_SERVER['PHP_AUTH_USER']]);
```

## Раздел 4. CSRF.

Добавлена защита от подделки запросов от имени авторизованного пользователя. При входе в систему генерируется уникальный ключ — `csrf_token`, который добавляется в скрытые поля форм. Я вставила его во все формы как скрытое поле. При получении POST или GET запроса сервер проверяет совпадение ключа. Если не совпадает — действие блокируется.

### Пример кода:

```
if (!isset($_POST['csrf_token']) || !hash_equals($_SESSION['csrf_token'], $_POST['csrf_token'])) {  
    die('CSRF token validation failed');  
}
```

## Раздел 5. Include и Upload уязвимости.

Проверка подключения файл в коде (например, корректности названий), чтобы избежать возможности подключения сторонних файлов на сервере или загрузке вредоносных скриптов.

### Пример использования:

1. *Include*: используется фиксированное подключение локальных файлов (`include('form.php')`). Параметры из URL не используются в функциях подключения файлов.
2. *Upload*: Пользователи не могут осуществлять загрузку файлов, что исключает возможность атаки через загрузку вредоносного ПО.